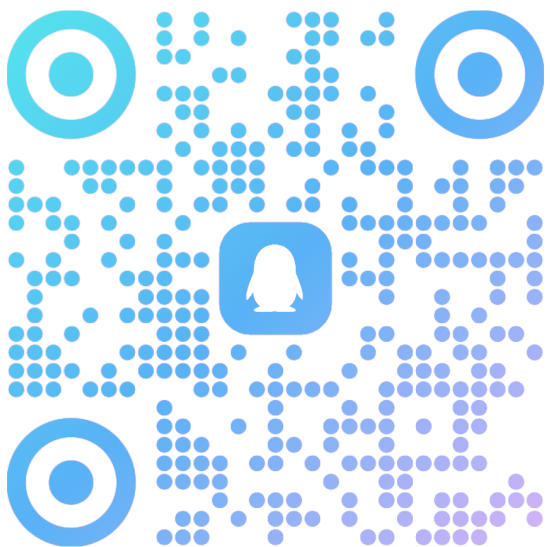




福州大学-2025编译...

群号: 1037351079



扫一扫二维码，加入群聊



编译原理

2025 春

任课教师: 柯嵩宇

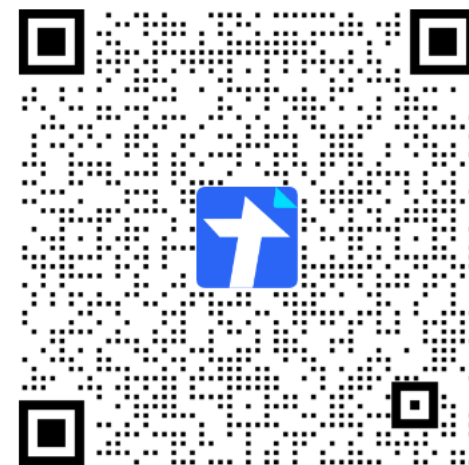
Email: songyuke@fzu.edu.cn

2025-03-20



3.20课堂

扫一扫二维码打开或分享给好友



— 腾讯文档 —

可多人实时在线编辑，权限安全可控

课前复习

编程语言的语法

- 数学定义

- $G = (V_N, V_T, S, P)$
- V_N 是**语法成分**，也称语法变量，必定在某个生成式左边出现
- V_T 是终结符号集合，**词法单元 (token)**，为源代码中具体的一个字符串，可以为空，只能在生成式**右边**出现
- S 是初始符号，最高层次的语法成分，对于编程语言来说，可以是**program**或者**module**
 - 通常是第1条产生式规则左部的文法符号
- P 是产生式的集合

- 产生式/推导式

$$A \rightarrow B|C|D|E|\dots$$

语法的推导(Derivation)

- 给定文法 $G = (V_T, V_N, P, S)$, 若 $A \rightarrow \gamma \in P$ 且 $\alpha, \beta \in (V_T \cup V_N)^*$, 则文法符号串 $\alpha A \beta$ 可重写为 $\alpha \gamma \beta$, 记作

$$\alpha A \beta \Rightarrow \alpha \gamma \beta,$$

称 $\alpha A \beta$ 直接推导 $\alpha \gamma \beta$ 。

- 如果 $\alpha_1 \Rightarrow \alpha_2 \Rightarrow \dots \Rightarrow \alpha_n$, 则称这个序列是从 α_1 到 α_n 的一个推导。
- 若存在一个从 α_1 到 α_n 的推导, 则称 α_1 可以推导出 α_n 。

推导的步数

- 推导长度：执行一步推导的次数
 - 一步推导
 - 每次只替换一个非终结符，用 $\Rightarrow$ 表示
 - 1步或若干步的推导
 - 用 $\Rightarrow^+$ 表示
 - 0步或若干步的推导
 - 用 $\Rightarrow^*$ 表示

文法、语言、句型、句子

- 文法定义的语言

- 从开始符号出发，利用推导能得到的所有终结符号串的集合

- 句型

- 如果 $S \Rightarrow^* \alpha, \alpha \in (V_T \cup V_N)^*$, 则 α 为文法 G 的句型

- 句子

- 如果 $S \Rightarrow^+ \alpha, \alpha \in V_T^*$, 则 α 为文法 G 的句子

S 是文法的起始符号

- 语言

- 文法 G 的所有句子集合成为**文法 G 生成的语言**，记为 $L(G)$

$$L(G) = \{\alpha \mid S \Rightarrow^+ \alpha, \alpha \in V_T^*\}$$

- 上下文无关文法生成的语言称为**上下文无关语言**

文法和语言

- 文法和语言的关系
 - 给定一个文法，就从结构上唯一地确定其语言
 - 给定一种语言，能确定其文法，但文法不唯一
- 等价文法
 - 设 G_1 和 G_2 是两个文法，若 $L(G_1) = L(G_2)$ ，称 G_1 与 G_2 为等价文法，可记作 $G_1 \equiv G_2$

语法分析树

- 从一个句型到一个句型的推导往往不唯一，可以以图形方式描述一个句子的推导过程，称为**语法分析树**（**语法树, Syntax Tree**）
- 某一文法的语法树具有以下性质：
 - **根节点**是**开始符号**
 - **叶节点**是**终结符**（即 token）或 ϵ
 - **内部节点**（非叶结点）是**非终结符**
 - 如果应用了规则 $A \rightarrow x_1x_2\dots x_n$ ，则
 - A 是内部结点，该子树的根节点
 - $x_1, x_2, \dots, x_n$ 是其从左到右的子结点（内部节点或叶节点）
 - 全部叶子节点从左到右构成**开始符号**推导得到的**符号串**

语法的二义性

- 文法的二义性
 - 若文法存在某个句子对应两棵不同的**语法树**，则称该文法是**二义的**（**Ambiguous**）
- 造成二义性的原因
 - 文法中没有体现出结合律和优先级
- 解决方案
 - 在文法中体现结合律和优先级

一种可行的无二义性约定

- 按照优先级从低到高
 - $expr_0 \rightarrow \dots | expr_1$
 - $expr_1 \rightarrow \dots | expr_2$
 - $expr_2 \rightarrow \dots | expr_3$
 - ...
 - $expr_{n-1} \rightarrow \dots | expr_n$
 - $expr_n \rightarrow num | variable | (expr_0)$
- 左结合
 - $expr_i \rightarrow expr_i \oplus expr_{i+1} | \dots | expr_{i+1}$
- 右结合
 - $expr_i \rightarrow expr_{i+1} \oplus expr_i | \dots | expr_{i+1}$

上下文无关文法与下推自动机

下推自动机 (Pushdown Automaton, PDA)

- 下推自动机，是一个抽象的机器用于识别和接受字符输入。
- 它和非确定型有限自动机类似，但多出一个称为栈 (stack) 的组件，且栈的规模是无限的。
- PDA默认都是非确定性的，确定性的和非确定性的不能等价。

下推自动机的数学定义

- PDA可以被定义为下面的六元组

$$M = (Q, \Sigma, \Gamma, \delta, q_0, F)$$

- Q 是所有状态的集合，有限个
- Σ 是输入字符的集合，有限。特别地， $\Sigma_\epsilon = \{\epsilon\} \cup \Sigma$
- Γ 是栈字符的集合，有限。特别地， $\Gamma_\epsilon = \{\epsilon\} \cup \Gamma$
- δ 是转移函数

$$\delta: Q \times \Sigma_\epsilon \times \Gamma_\epsilon \rightarrow Q \times \Gamma_\epsilon$$

- 根据输入和当前栈顶元素进行转移的同时替换栈顶元素
- 特别地，转移函数支持 ϵ 作为栈顶元素，也支持将栈顶元素替换为 ϵ 。这两个操作一个是单纯压栈，另一个是单纯弹栈
- q_0, F 的意思和NFA中的一致

接受 $\{0^n 1^n \mid n \geq 1\}$ 的PDA

接受 $\{0^n 1^n \mid n \geq 0\}$ 的 PDA

上下文无关文法与PDA

上下文无关文法与PDA

- 定理

- 一个语言是上下文无关的，当且仅当存在一个下推自动机识别该语言。

- 引理

- 若一个语言是上下文无关的，则存在一个下推自动机识别能够识别该语言。
- 根据一个上下文无关文法能够构造出一个对应的下推自动机。
- 若一个语言可以被PDA接受，它便是一个上下文无关语言。

最左推导/最右推导

最左推导/最右推导

- 最左推导 (Leftmost derivation)
 - 首先替换最左边的非终结符
 - 最右推导 (Rightmost derivation)
 - 首先替换最右边的非终结符
 - 最左 (右) 推导与分析树一一对应
 - 可以通过说明对于某一个串存在两个不同的最左推导来证明文法二义
 - 可以通过说明对于某一个串存在两个不同的最右推导来证明文法二义
- 语法分析树的结构与使用最左推导还是最右推导无关**

最左推导/最右推导

- 文法

$G(S): S \rightarrow AB, A \rightarrow aaA, A \rightarrow \varepsilon, B \rightarrow b$

- 推导出句子: aab

- 最左推导:

- $S \Rightarrow AB \Rightarrow aaAB \Rightarrow aaB \Rightarrow aab$

- 最右推导:

- $S \Rightarrow AB \Rightarrow Ab \Rightarrow aaAb \Rightarrow aab$

练习

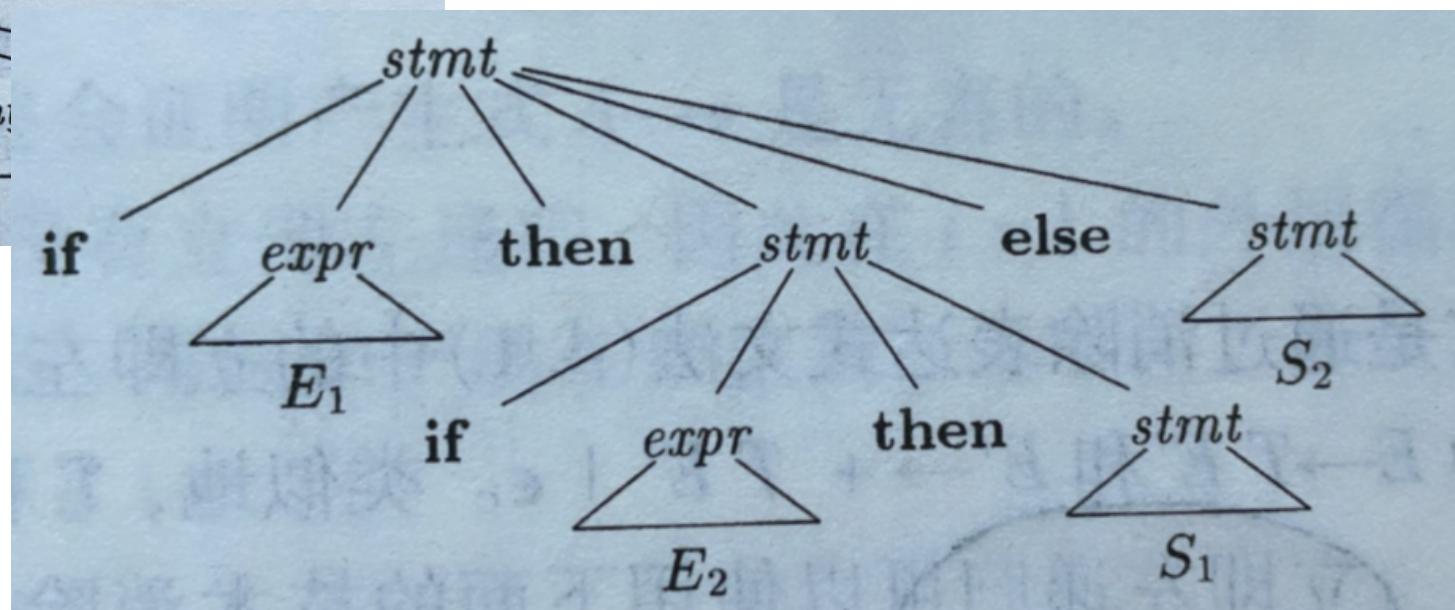
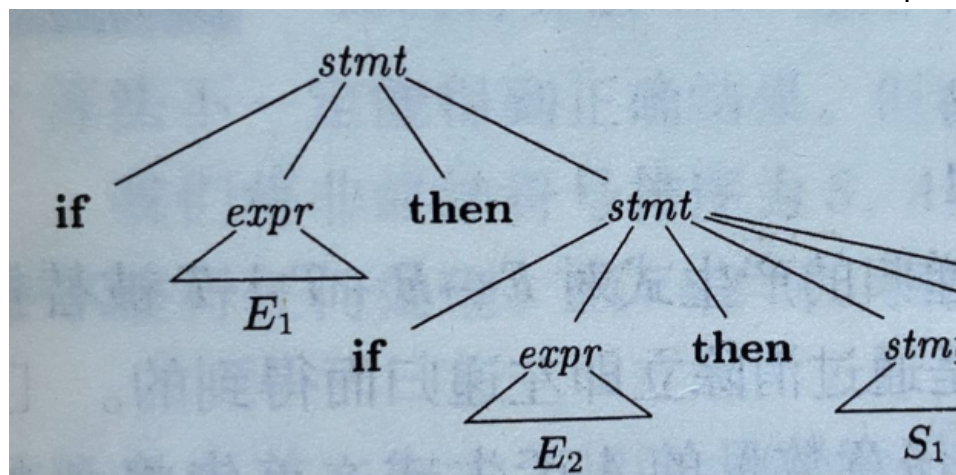
- 证明文法 $S \rightarrow aSbS \mid bSaS \mid \varepsilon$ 是二义的
- 第二种证明方法——给出两个不同的最左推导
 - $S \Rightarrow aSbS \Rightarrow abSaSbS \Rightarrow abaSbS \Rightarrow ababS \Rightarrow abab$
 - $S \Rightarrow aSbS \Rightarrow abS \Rightarrow abaSbS \Rightarrow ababS \Rightarrow abab$

改写二义性文法

- 巴科斯范式（Backus Normal Form, BNF），是一种用于表示上下文无关文法的语言
 - $\langle \text{符号} \rangle ::= \langle \text{使用符号的表达式} \rangle$
 - $\langle \text{符号} \rangle$ 是**非终结符**
 - 表达式由
 - 一个符号序列
 - 或用竖杠 '|' 分隔的多个符号序列构成
 - 每个符号序列整体都是左端的符号的一种可能的替代
 - 从未在左端出现的符号叫做**终结符**

改写二义性文法

```
stmt ::= if (expr) then stmt  
       | if (expr) then stmt else stmt  
       | others
```



改写二义性文法

```
stmt ::= if (expr) then stmt  
      | if (expr) then stmt else stmt  
      | others
```

```
stmt  
  ::= matched_stmt | unmatched_stmt  
matched_stmt  
  ::= if (expr) then matched_stmt else matched_stmt  
      | other_stmt  
unmatched_stmt  
  ::= if (expr) then stmt  
      | if (expr) then matched_stmt else unmatched_stmt
```

练习

- 用BNF范式书写以下表达式文法
 - 运算符: +, -, *, /, <<, >>, <, >, <=, >=, ==, !=
 - 优先级从高到低
 1. 乘除: *, /
 2. 加减: +, -
 3. 位移运算: <<, >>
 4. 比较运算: >, <, >=, <=
 5. 相等比较: ==, !=
 - 算符均为左结合
 - 操作数为单个小写字母的变量名

练习

expression ::= equality

equality ::= equality == comparison | equality != comparison | comparison

comparison ::= comparison < shift | comparison <= shift | comparison >
shift | comparison >= shift | shift

shift ::= shift << term | shift >> term | term

term ::= term + factor | term - factor | factor

factor ::= factor / primary | factor * primary | factor % primary | primary

primary ::= variable | (expression)

variable ::= a|b|c|...|z

语法分析器的设计

自顶向下(Top-Down)

- 从文法的**开始符号**出发，反复使用各种产生式，寻找"匹配"的**推导**
- 推导：根据文法的产生式规则，把串中出现的产生式的左部符号替换成右部
- **从树的根开始**，构造语法树
- 递归下降分析法、**预测分析法**

自底向上(Bottom-Up)

- 从**输入串**开始，逐步进行**归约**，直到文法的开始符号
- 归约：根据文法的产生式规则，把串中出现的产生式的右部替换成左部符号
- **从树叶节点开始**，构造语法树
- 算符优先分析法、**LR分析法**

自顶向下分析

- 目的
 - 给定文法 G 和句子 s , 回答 s 是否能从 G 推导出来?
- 基本思想
- 挑战
- 递归下降分析法
- 预测分析法

自顶向下分析的基本思想

- 从根节点向叶节点方向构造分析树，即从文法的开始符号S出发推导出词串s的过程
- 它不能处理左递归文法
- 基本思想
 - 总是选择每个句型的最左非终结符进行替换
 - 根据输入流中的下一个终结符，选择最左非终结符的一个候选式

挑战

- 无限循环
 - 原因：左递归
 - 直接左递归
 - 间接左递归
 - 解决方法
 - 重写文法
 - 消除左递归
- 回溯
 - 原因：匹配失败
 - 解决方法
 - 提取左公因式

问题一：左递归

文法

- $\text{expr} ::= \text{expr} + \text{term} \mid \text{expr} - \text{term} \mid \text{term}$
- $\text{term} ::= \text{term} * \text{factor} \mid \text{term} / \text{factor} \mid \text{factor}$
- $\text{factor} ::= \text{digit} \mid (\text{expr})$
- $\text{digit} ::= 0 \mid 1 \mid \dots \mid 9$

8+8+8+8

expr

- expr + term
- expr + term + term
- expr + term + term
- expr + term + term + term
- expr + term + term + term + term
- expr + term + term + term + term + term
- ...

问题一：左递归 (Left-recursive)

直接左递归 (Direct Left-recursive)

- $\text{expr} ::= \text{expr} + \text{term}$

间接左递归 (Indirect Left-recursive)

- $\text{expr} := \text{term} + \text{term}$
- $\text{term} := \text{factor} * \text{factor}$
- $\text{factor} := \text{expr}$

消除直接左递归

- $\text{expr} ::= \text{expr} + \text{term} \mid \text{expr} - \text{term} \mid \text{term}$
- $\text{term} ::= \text{term} * \text{factor} \mid \text{term} / \text{factor} \mid \text{factor}$
- $\text{factor} ::= \text{digit} \mid (\text{expr})$

- $\text{expr} ::= \text{term expr}_1$
- $\text{expr}_1 ::= + \text{term expr}_1 \mid - \text{term expr}_1 \mid \varepsilon$
- $\text{term} ::= \text{factor term}_1$
- $\text{term}_1 ::= * \text{factor term}_1 \mid / \text{factor term}_1 \mid \varepsilon$
- $\text{factor} ::= \text{digit} \mid (\text{expr})$

消除间接左递归

1. 按照某个顺序将非终结符号排序为 $A_1, A_2, \dots, A_n$.
2. for i in $[1, 2, \dots, n]$ do
3. for j in $[1, 2, \dots, i]$ do
4. **消除由 A_j 导致的左递归**, 即向每个形如 $A_i \rightarrow A_j \gamma$ 的产生式代入所有 A_j 为头的规则, 得到 $A_i \rightarrow \delta_1 \gamma \mid \delta_2 \gamma \mid \dots \mid \delta_k \gamma$, 其中 $A_j \rightarrow \delta_1 \mid \delta_2 \mid \dots \mid \delta_k$, 是所有的 A_j 产生式
5. endfor
6. **消除 A_i 产生式之间的直接左递归**
7. endfor

问题二：回溯

- 分析过程中，当一个非终结符用某一个候选式匹配成功时，这种匹配可能是暂时的。**出错时**，需要进行“回溯”
- 当同一个非终结符的多个候选式存在**共同前缀**时，将导致**回溯**。

回溯问题示例

- $G(S)$:
 - $S \rightarrow xAy$
 - $A \rightarrow * * \mid *$
- 分析 $x * y$

提取左公因式

1. 对于每个非终结符 A ，找出它的两个或多个候选式的最长公共前缀 α 。若 $\alpha \neq \varepsilon$ ，则将所有 A 产生式

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \dots \mid \alpha\beta_n \mid \gamma_1 \mid \gamma_2 \mid \dots \mid \gamma_m$$

替换为

$$A \rightarrow \alpha A' \mid \gamma_1 \mid \gamma_2 \mid \dots \mid \gamma_m$$

$$A' \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

其中， γ_i 表示所有不以 α 开头的产生式体；

2. 不断应用这个转换，直到每个非终结符的任意两个产生式体都没有公共前缀为止

示例

```
stmt ::= if (expr) then stmt  
      | if (expr) then stmt else stmt  
      | others
```

stmt

```
::= matched_stmt | unmatched_stmt
```

matched_stmt

```
::= if (expr) then matched_stmt else matched_stmt  
   | other_stmt
```

unmatched_stmt

```
::= if (expr) then stmt  
   | if (expr) then matched_stmt else unmatched_stmt
```

示例

stmt

::= matched_stmt | unmatched_stmt

matched_stmt

::= if (expr) then matched_stmt else matched_stmt
 | other_stmt

unmatched_stmt

::= if_then unmatched_rest

unmatched_rest

::= matched_stmt | unmatched_stmt | matched_stmt else unmatched_stmt

if_then

::= if (expr) then

示例

```
stmt
    ::= matched_stmt | unmatched_stmt
matched_stmt
    ::= if (expr) then matched_stmt else matched_stmt
        | other_stmt
unmatched_stmt
    ::= if_then unmatched_rest
unmatched_rest
    ::= unmatched_rest_prefix_0 unmatched_rest_suffix_0 | unmatched_stmt
unmatched_rest_prefix_0
    ::= matched_stmt
unmatched_rest_suffix_0
    ::= else unmatched_stmt |  $\epsilon$ 
if_then
    ::= if (expr) then
```

递归下降的语法分析

- 自顶向下
 - 从分析树的根结点开始，深度优先地创建分析树的各个结点
 - 最左推导
- 递归下降分析程序
 - 由一组过程组成，每个非终结符号有一个对应的过程，识别对应的语法单位，当需要从某个非终结符出发进行展开(推导)时，就调用这个非终结符对应的子程序。
- 程序的执行从开始符号对应的过程开始，如果这个过程的过程体扫描了整个输入串，它就停止执行并宣布语法分析成功。

